Pass 1 Extension Labeling

理解扩展标注、读懂 example、稳妥新增自己的 spec

`trigger → extra labels → dashboard / review export`

先记住一句话

扩展标注 = 在核心 Pass 1 9 维标签之上，额外增加一层“领域专用标签”。

- 它在 **核心 Pass 1 完成之后** 才运行
- 它会增加 `label_extensions.<spec_id>` 结果
- 它 **不会修改** 核心 9 维 taxonomy
- 它 **不是** Pass 2 质量评分器

> *扩展是“额外 metadata”，不是“重写核心标注”。*

最推荐的理解路径

```
raw sample
  ↓
core Pass 1 labeling
  ↓
extension trigger check
  ↓
matched ? extra extension call : skipped
  ↓
label_extensions.<spec_id>
  ↓
dashboard / stats_labeling.json / export-review --include-extensions
```

- trigger 判断依赖 **核心 Pass 1 labels**
- 未命中 trigger 是正常现象，不是报错

什么时候该用扩展？什么时候不该？

适合用

- 想给某个子集增加领域标签
- 想分析数据配比 / 覆盖面
- 想给 review CSV 增加额外列
- 想并行维护 Web / mobile / domain-specific 视角

不适合用

- 想“修正”核心 Pass 1 标签
- 想直接做回答质量评分
- 还没过滤/采样，就想对全量数据开强领域 spec
- 把多个完全不同领域强塞进一个 spec

怎么启用它

交互式

```
uv run sft-label start
```

- 先开启 `Enable extension labeling`
- 然后：
  - 选默认目录 `extensions/`
  - 或手动输入单个 YAML
  - 或输入一个目录后多选

CLI

```
uv run sft-label run \
  --input data.jsonl \
  --label-extension /path/to/spec_a.yaml \
  --label-extension /path/to/spec_b.yaml
```

> `extensions/` *只是 launcher 默认查找 spec 的位置，**不会自动运行其中所有文件。***

一个 extension spec 里有什么

```yaml
id: my_extension
spec_version: v1
trigger:
  domain_any_of: [web-frontend]
prompt: |
  Return JSON only.
schema:
  field_a:
    type: enum
    options: [x, y, z]
output:
  include_confidence: true
  allow_unmapped: true
```

顶层块怎么理解

- `id`：存储键，必须唯一
- `trigger`：什么时候跑
- `prompt`：这层扩展的额外指令
- `schema`：必须输出的 JSON 结构

trigger 语义：第一次用最容易误解的地方

核心规则

- trigger 是在 **核心 Pass 1 完成后** 判断的
- 它匹配的是核心 labels：`domain / language / intent / task / context / difficulty`
- **不是** 直接对原始文本做关键词匹配

列表语义

- `language_any_of: []` = **这个维度不参与过滤**
- 非空 `*_any_of` = 这个维度变成 **硬前置条件**
- 同一维度内多个值 = OR
- 不同 trigger 维度之间 = AND

未命中时

- `status=skipped`
- `matched=false`

> `skipped` *通常是正常路由结果，不代表失败。*

成本模型一定要先懂

每命中一个 extension，就会为该样本增加一次额外 extension-labeling 调用。

简单估算：

```
extra calls ≈ 各 extension 的 matched samples 总和
```

例如：

- spec A 命中 1,000 条
- spec B 命中 1,000 条
- 则大约增加 **2,000** 次额外调用

含义

- trigger 越宽，召回越高，但调用数 / 时延 / review 成本越高
- trigger 越严，成本更低，但更容易“静默跳过”
- **第一轮一定先过滤 / 采样 / limit**

自适应运行时和扩展标注的关系

- 默认开启 **adaptive runtime**
- `--concurrency` 和 `--rps-limit` 是 **上限 caps**
- 并不保证运行时会一直顶到这个数
- 如果模型服务有压力、限速、抖动：
  - 实际速度可能下降
  - 会暂时退让
  - `recovery sweep` 只是在阶段末做一轮保守补跑，不是全量重跑

> *扩展开多了以后，调用量会变大，所以更容易感知到 adaptive runtime 的作用。*

结果会写到哪里

每条样本

- `label_extensions.<spec_id>`

聚合统计

- `stats_labeling.json`
- `extension_stats.specs.<spec_id>`

导出审核

```
uv run sft-label export-review \
  --input <run_dir> \
  --output review.csv \
  --include-extensions
```

dashboard

- 可以看 matched / skipped / invalid / unmapped
- 可以看字段分布、低置信度、抽样明细

仓库自带 example：它是干什么的

默认 starter：

- `extensions/ui_web_analysis_v1.example.yaml`

参考副本：

- `docs/examples/extensions/ui_web_analysis_v1.yaml`

这个 example 的定位

- 用来分析 **Web / desktop browser UI 数据配比**
- 用来发现：
  - 是否过度集中在 CRUD / 表单
  - 是否缺少 dashboard / builder / 管理台等 richer workflows
- **不是** 用来评价回答质量

这个 example 为什么只开了 `domain_any_of`

```yaml
trigger:
  domain_any_of: [web-frontend]
  language_any_of: []
  intent_any_of: []
  task_any_of: []
  context_any_of: []
  difficulty_any_of: []
```

设计意图

- 默认先走 **分析导向 / recall-first**
- 只保留 broad Web domain trigger
- 其他维度留空，是为了：
  - 让用户看见可用路由维度
  - 先不要过早缩窄召回

这代表什么

- 如果你把其他列表填成非空
- 就表示你开始 **更强地相信核心 Pass 1 路由精度**

这个 example 的 5 个字段分别在回答什么问题

字段	它想回答什么	它帮助你发现什么失衡
`ui_surface_type`	这是哪类 Web UI 表面?	数据是否被单一页面形态垄断
`interaction_pattern`	主要交互模式是什么?	是否过度集中在 CRUD / 表单
`state_data_complexity`	状态/数据协调难度如何?	是否缺少 async / 多源 / 表单联动样本
`ui_constraint_focus`	显著的 UI 工程约束是什么?	design system / responsive / a11y 是否稀缺
`frontend_stack_shape`	主要生态形态是什么?	框架/栈是否过度偏斜

> *这些字段是为 **数据分析 / 配比优化** 服务，不是给每条样本增加更多“任务要求”。*

读 example YAML 时，建议这样读

从上到下

1. `id` / `display_name`：这个 spec 叫什么
2. `trigger`：它准备拦哪些样本
3. `prompt`：它要模型从什么角度输出 JSON
4. `schema`：有哪些字段、每个字段允许什么值
5. `output`：是否带 confidence / unmapped
6. `dashboard`：只是展示分组和优先级，不改变标注逻辑

额外提醒

- prompt 可以是中文，也可以是英文
- 真正重要的是：**输出 JSON 必须匹配 schema**

从 example 克隆自己的 extension：最稳做法

建议顺序

1. 复制 `extensions/ui_web_analysis_v1.example.yaml` 到新文件
2. **先改 `id`**
3. 再改 `display_name`
4. 再改 `prompt`
5. 最后改 `schema` / `trigger`

第一版推荐

- 先控制在 **3–5 个字段**
- 一个 spec 只覆盖 **一个领域**
- Web 和 mobile **拆成两个 spec**

> *对第一次上手用户，**复制到新文件** 比“直接原地改 example”更稳。*

新建自己的 spec：哪些必须先改，哪些先别动

必须先改

- `id`
- `display_name`
- `prompt`
- `schema`

通常先保留

- `output.include_confidence`
- `output.allow_unmapped`

后面再决定

- 更严的 trigger
- `dashboard.group`
- `dashboard.priority`

推荐首轮 rollout

1. 复制 example 到新文件
2. 先用过滤后子集、小样本或 `--limit 20/50/100`
3. 看 `matched / skipped`
4. 看 dashboard 字段分布
5. 抽查 10—20 条样本的 `label_extensions.<id>`
6. 跑一次 `export-review --include-extensions`
7. 没问题，再扩大范围或加字段

你真正要检查的不是“有没有结果”

- 而是结果是否 **可解释、可复核、可稳定扩展**

多个 extension 可以同时开，但它们彼此独立

- 同一条样本可以同时命中多个 extension
- 每个 extension 的结果独立写在：
  - `label_extensions.<spec_id>`
- 一个 extension 可以 matched
- 另一个 extension 同时 skipped / failed
- **重复 `id` 不允许**

什么时候该拆多个

- Web vs mobile
- analysis vs domain heuristics
- 不同 trigger 逻辑
- 字段太多 / options 太多 / prompt 太长

一页记住：安全默认实践

- 先用 **一个小 spec**
- 先跑 **一个小样本**
- 先看 **matched / skipped / low-confidence / unmapped**
- 先用 dashboard 和 review export 做抽查
- Web / mobile 分开
- 不要太早把 trigger 收得很窄
- 不要把强领域 spec 直接跑到全量数据

继续看

- 长文指南：`docs/guides/pass1-extension-labeling.md`
- launcher 指南：`docs/guides/interactive-launcher.md`
- 默认 example：`extensions/ui_web_analysis_v1.example.yaml`